

Towards Automatic Generation of Vulnerability-Based Signatures

David Brumley, James Newsome, and Dawn Song
Carnegie Mellon University
Pittsburgh, PA, USA
{dbrumley,jnewsome,dawnsong}@cmu.edu

Hao Wang and Somesh Jha
University of Wisconsin-Madison
Madison, WI, USA
{hbwang, jha}@cs.wisc.edu

Abstract

In this paper we explore the problem of creating vulnerability signatures. A vulnerability signature matches all exploits of a given vulnerability, even polymorphic or metamorphic variants. Our work departs from previous approaches by focusing on the semantics of the program and vulnerability exercised by a sample exploit instead of the semantics or syntax of the exploit itself. We show the semantics of a vulnerability define a language which contains all and only those inputs that exploit the vulnerability. A vulnerability signature is a representation (e.g., a regular expression) of the vulnerability language. Unlike exploit-based signatures whose error rate can only be empirically measured for known test cases, the quality of a vulnerability signature can be formally quantified for all possible inputs.

We provide a formal definition of a vulnerability signature and investigate the computational complexity of creating and matching vulnerability signatures. We also systematically explore the design space of vulnerability signatures. We identify three central issues in vulnerability-signature creation: how a vulnerability signature represents the set of inputs that may exercise a vulnerability, the vulnerability coverage (i.e., number of vulnerable program paths) that is subject to our analysis during signature creation, and how a vulnerability signature is then created for a given representation and coverage.

We propose new data-flow analysis and novel adoption of existing techniques such as constraint solving for automatically generating vulnerability signatures. We have built a prototype system to test our techniques. Our experiments show that we can automatically generate a vulnerability signature using a single exploit which is of much higher quality than previous exploit-based signatures. In addition, our techniques have several other security applications, and thus may be of independent interest.

1 Introduction

A *vulnerability* is a type of bug that can be used by an attacker to alter the intended operation of the software in a malicious way. An *exploit* is an actual input that triggers a software vulnerability, typically with malicious intent and devastating consequences. One of the most popular and effective exploit defense mechanisms is signature-based input filtering (also called content-based filtering) [7, 40]. Thus, any improvements in signature generation will likely have widespread impact.

We need *automatic* signature generation techniques because manual signature generation is slow and error prone. Fast generation is important because previously unknown (“zero-day”) or unpatched vulnerabilities can be exploited orders of magnitude faster than a human can respond, such as during a worm outbreak [7, 51]. Automatic techniques have the potential to be more accurate than manual efforts because vulnerabilities tend to be complex and require intricate knowledge of details such as realizable program paths and corner conditions. Understanding the complexities of a vulnerability has consistently proven very difficult for humans at even the source code level [11], let alone COTS software at the assembly level.

Challenges for automatically creating signatures. The task of automatically constructing signatures is complicated by the fact that there are usually several different *polymorphic* exploit variants that can trigger a software vulnerability [21, 33, 43]. For example, a buffer-overflow vulnerability in a network service may be triggered by many different protocol messages. Another example, sometimes referred to as *metamorphism*, is that exploit variants may differ syntactically but be semantically equivalent [28, 52], e.g., an exploit could use different assembly instructions that have the same effect. Our approach does not need to distinguish between polymorphism and metamorphism: both are referred to as polymorphism throughout this paper. Many morphing tools are publicly available to automatically generate polymorphic exploit variants [2, 3, 21]. Thus, to be

effective, the signature should be constructed based on the property of the vulnerability, instead of an exploit (this observation has been made by others as well [56]).

Limitations of previous approaches. The importance of the signature generation problem has recently prompted researchers to investigate automatic signature generation techniques. Previous approaches fall into at least one of the following categories: (a) require manual steps, (b) employ heuristics which may fail in many settings, (c) techniques rely on specific properties of an exploit, e.g., return addresses, and are thus not *vulnerability* signatures, (d) are limited by the underlying signature representation they can generate, or (e) only work for specific vulnerabilities in specific circumstances.

For example, one approach is pattern-extraction based methods which syntactically identify bit patterns that appear in attack samples but not in innocuous samples [30, 32, 43, 50]. However, these techniques are either incapable of handling polymorphic worms [30, 32, 50], or vulnerable in an adversarial environment in which an attacker can inject false or superfluous tokens, such as an over-learning or “red-herring” attack [43]. Another approach is based on application and exploit semantic information [35, 44, 58]. However, these techniques are heuristics-based and rely on specific properties of the exploits such as the value used to overwrite the return address to be invariant. It has been shown previously these heuristics may not work in many real-world vulnerabilities [19, 43]. In addition, previous work along either line has not systematically explored the design space of signature creation, instead focusing on a single design point such as creating regular expressions for control-hijacking attacks. Regular-expressions can only recognize simple syntactic properties, thus may not be precise enough in many settings, e.g., regular expressions cannot recognize a vulnerability where valid and invalid checksums need to be distinguished.

Our approach, roadmap, and the central issues. Our approach departs from previous work by analyzing the vulnerability uncovered by a new exploit attack instead of analyzing the exploit. At a high level, our main contribution is a new class of signature, which we call a *vulnerability signature*, that is not specific to details such as whether an exploit successfully hijacks control of the program, but instead whether executing an input will (potentially) result in an unsafe execution state.

In this paper we present a formal approach for reasoning about vulnerability signatures. Intuitively, a vulnerability signature matches a *set of inputs* (strings) which satisfy a *vulnerability condition* in the program. A vulnerability condition is a specification of a particular type of program bug, e.g., memory writes should be within the allocated buffer

space. We then systematically explore the design space of vulnerability signatures, and identify two important dimensions: how the signature is *represented*, in which there is an expressiveness trade-off between matching accuracy and efficiency, and how much of the vulnerability is *covered* by the signature, in which there is a trade-off between the amount of analysis performed and the signature false negative rate. We then develop new techniques for creating vulnerability signatures for different representations. We focus on three representations which highlight the inherent accuracy, efficiency, and creation time trade-offs in the design space: *Turing machine* signatures, *symbolic constraint* signatures, and *regular expression* signatures.

Contributions. This paper presents a systematic approach using a formal model and methods to create vulnerability signatures using static program analysis. We require only a single sample exploit which is used to initially identify the vulnerability. Our automatic signature generation approach is applicable to all vulnerabilities in which the vulnerability condition can be formally specified. Our approach uncovers a rich new domain for representing signatures and new techniques for creating them. In particular:

- We provide a formal definition for vulnerability signatures. Our approach leads to a new perspective where a vulnerability signature can be represented by different language classes with different expressive powers.
- We explore the design space of vulnerability signature and show that there is an inherent trade-off between signature matching and accuracy for different representations. In particular, a perfect signature can be created (Turing machine signatures in Section 2.3), but matching may take an unbounded amount of time. On the other hand, signatures that allow fast matching are less accurate (regular-expression signatures in Section 2.3).
- We introduce the notion of vulnerability signature coverage. As we will see, one challenge is that a vulnerability may be reachable by an infinite number of paths in the program (in the presence of looping). We show how to iteratively consider each path separately so that signature generation can scale.
- Our methods allow us to identify where a created vulnerability signature approximates a perfect vulnerability signature. Specifically, in our setting, one can identify and control when and how imprecision is introduced. This property makes it easy to quantify the quality of the generated vulnerability signature.
- We develop new static analysis techniques (such as the regular expression data-flow framework in Section 3.4.2), and make novel adoptions of existing techniques such as program chopping and constraint satisfaction to our problem domain.

- We provide a prototype implementation of our techniques and automatically create signatures for several real-world vulnerabilities. Our prototype addresses automatic signatures creation in one of the hardest scenarios: only the program binary is used. We do not require source code or type information, and therefore our prototype is applicable to COTS software.
- Our results show that our techniques automatically generate signatures that are of a much higher quality than previous techniques.

2 Vulnerability Signature

In this section we first give a formal definition of a vulnerability signature. Intuitively, a vulnerability signature is a representation for the set of inputs that satisfy a specified vulnerability condition (vulnerability conditions are formally defined in Section 2.2). We then explore two dimensions of the design space for vulnerability signatures: signature representation and coverage. Roughly speaking, design points in the signature representation dimension trade-off matching accuracy and matching efficiency. Design points in the vulnerability signature creation dimension trade-off creation time for signature coverage, i.e., how many program paths are analyzed.

Problem setting. We motivate our work and approach to vulnerability signatures in the following setting: a new exploit is just released for an unknown vulnerability. A site has detected the exploit through some means such as dynamic taint analysis or stack protection, and wishes to create a signature that recognizes any further exploits. The site can furnish our analysis with the tuple $\{\mathcal{P}, T, x, c\}$ where $\mathcal{P}$ is the program, x is the exploit string, c is a vulnerability condition, and T is the execution trace of $\mathcal{P}$ on x . Since our experiments are at the assembly level, we assume $\mathcal{P}$ is a binary program and T is an instruction trace, though our techniques also work at the source-code level. Our goal is to create a vulnerability signature which will *match* future malicious inputs x' by examining them without running $\mathcal{P}$.

In addition, we want to create signatures quickly since in many scenarios signatures must be deployed almost immediately after detection to be of any value. Therefore, we take an iterative approach that generates successively better signatures. Each successive signature will match more exploit variants *without requiring further exploit samples*.

Running example. Throughout this paper, we use the running example given in Figure 1. Our example is in a C-like language for clarity; our implementation operates on program binaries. The example returns the URL when the request begins with the 'G' or 'g' keyword, else NULL is returned. In our example, we will

```

1 char *get_url(char inp[10]){
2     char *url = malloc(4);
3     int c = 0;
4     if(inp[c] != 'g' && inp[c] != 'G')
5         return NULL;
6     inp[c] = 'G';
7     c++;
8     while(inp[c] == ' ')
9         c++;
10    while(inp[c] != ' '){
11        *url = inp[c]; c++; url++;
12    }
13    printf('%s', url);
14    return url;
15 }
```

Figure 1. Our running example, which returns the URL of a request of the form $[g|G]<url>$, else NULL.

assume $x = g /AAAA$. The corresponding trace is $T = \{1, 2, 3, 4, 6, 7, 8, 9, 8, 10, 11, 10, 11, 10, 11, 10, 11\}$ where each number is the corresponding line number in Figure 1. The vulnerability condition is a heap overflow, which the input x satisfies (i.e., the program is exploited) on the 5th iteration of line 11 since the URL is 5 characters long while only 4 characters were allocated.

2.1 Vulnerability Signature Definition

A *vulnerability* is 2-tuple $(\mathcal{P}, c)$, where $\mathcal{P}$ is a program (which is a sequence of instructions $\langle i_1, \dots, i_k \rangle$), and c is a vulnerability condition (defined formally below). The execution trace obtained by executing a program $\mathcal{P}$ on input x is denoted by $T(\mathcal{P}, x)$. An execution trace is simply a sequence of actual instructions that are executed. A vulnerability condition c is evaluated on an execution trace T . If T satisfies the vulnerability condition c , we denote it by $T \models c$. The *language* of a vulnerability $L_{\mathcal{P}, c}$ consists of the set of all inputs x to a program $\mathcal{P}$ such that the resulting execution trace satisfies c . Let Σ^* be the domain of inputs to $\mathcal{P}$. Formally, $L_{\mathcal{P}, c}$ is the language defined by:

$$L_{\mathcal{P}, c} = \{x \in \Sigma^* \mid T(\mathcal{P}, x) \models c\}$$

An *exploit* for a vulnerability $(\mathcal{P}, c)$ is simply an input $x \in L_{\mathcal{P}, c}$, i.e., executing $\mathcal{P}$ on input x results in a trace that satisfies the vulnerability condition c . A *vulnerability signature* is a matching function MATCH which for an input x returns either EXPLOIT or BENIGN without running $\mathcal{P}$. A *perfect* vulnerability signature satisfies the following

property:

$$\text{MATCH}(x) = \begin{cases} \text{EXPLOIT} & \text{when } x \in L_{\mathcal{P},c} \\ \text{BENIGN} & \text{when } x \notin L_{\mathcal{P},c} \end{cases}$$

As we show in Section 2.3, the language $L_{\mathcal{P},c}$ can be represented in many different ways ranging from Turing machines which are precise, i.e., accept exactly $L_{\mathcal{P},c}$, to regular expressions which may not be precise, i.e., have an error rate.

Soundness and completeness for signatures. We define completeness for a vulnerability signature MATCH to be $\forall x : x \in L_{\mathcal{P},c} \Rightarrow \text{MATCH}(x) = \text{EXPLOIT}$, i.e., MATCH accepts everything $L_{\mathcal{P},c}$ does. Incomplete solutions will have false negatives. We define soundness as $\forall x : x \notin L_{\mathcal{P},c} \Rightarrow \text{MATCH}(x) = \text{BENIGN}$, i.e., MATCH does not accept anything extra not in $L_{\mathcal{P},c}$.¹ Unsound solutions will have false positives. A consequence of Rice’s theorem [26] is that no signature representation other than a Turing machine can be both sound and complete, and therefore for other representations we must pick one or the other. In our setting, we focus on soundness, i.e., we tolerate false negatives but not false positives. In Section 5 we show how to reformulate our algorithm to generate complete but unsound signatures.

2.2 Vulnerability Conditions

The vulnerability condition c is a function which takes an instruction $I \in T$ and the current program state and returns either **EXPLOIT**, indicating $T \models c$, or **BENIGN** and a new program state reflecting the execution of I . The first instruction I such that $c(I) = \text{EXPLOIT}$ is called the *vulnerability point*. Intuitively, the vulnerability point is the first instruction which may cause unsafe execution, e.g., the first out-of-bounds write on line 11 of our running example.

Formally, a vulnerability condition c is a function

$$c : \Gamma \times D \times M \times K \times I \rightarrow \{\text{BENIGN}, \text{EXPLOIT}\}$$

where Γ is memory (including the state variables) for the vulnerability condition, D is the set of variables defined, M is the program’s map from memory locations to values, K is the continuation stack, and I is the next instruction to execute.² Formally, we execute c on each instruction in order of the trace T because each instruction could affect vulnerability condition state variables in Γ . In our scenario, Γ is a local memory for the vulnerability detection algorithm that may keep track of important variables such as bounds on allocated memory, D binds values to registers, K is the

¹Normally soundness is $\forall x : x \in S \Rightarrow x \in L_{\mathcal{P},c}$. Here we are stating the equivalent contra-positive.

²We note that Γ is strictly not necessary: it is only convenient to assume the vulnerability condition has memory separate from M .

evaluation stack (e.g., CISC instructions may dereference a calculated memory address all in the same instruction), and $M : \text{ADDR} \rightarrow \text{VALUE}$ is a map from 32-bit memory locations to values (including both stack and heap addresses).

We can encode c as an algorithm, and as described in Section 3 inline the encoding into the original program at the vulnerability point during signature creation. Also note that c need only be specified once for each type of vulnerability. Our contribution is not how to specify the vulnerability condition: we assume it is given. We note entire programming languages are specified in a similar manner to c (e.g., via formal operational semantics [45]), thus our techniques should apply to any vulnerability condition that can be stated with an algorithm.

In our running example the vulnerability condition is to check each dereference to make sure it is within the allocated bounds. One way to accomplish this is to shadow each pointer $\in T$ with a “safe” pointer value that records the base address and size of the memory allocated. Then, each dereference is checked to see if the corresponding safe pointer would still be in bounds. A formal operational semantics of this vulnerability condition may look like:

$$\begin{aligned} \Gamma, D, M, K \vdash * \text{exp} \rightsquigarrow D, M, K \triangleright * \square \vdash \text{exp} \\ \Gamma[n \rightarrow \text{SafePtr}(m, s)], D, M : [n \rightarrow v_n], K \triangleright * \square \vdash n \\ \rightsquigarrow \begin{cases} \text{BENIGN} & \text{if } m \leq n < m + s \\ \text{EXPLOIT} & \end{cases} \end{aligned}$$

The first rule says in order to calculate a memory dereference of the form $* \text{exp}$, exp must first be evaluated. Once exp is resolved to an address n , the second rule says to lookup n in the context Γ and get a safe pointer SafePtr . A safe pointer contains a base address m and a size s . If the dereferenced value is within the range specified, **BENIGN** is returned, else **EXPLOIT**.

2.3 Signature Representation Classes

We explore the space of different language classes that can be used to represent $L_{\mathcal{P},c}$ as a vulnerability signature. Which signature representation we pick determines the precision and matching efficiency. We investigate three concrete signature representations which reflect the intrinsic trade-offs between accuracy and matching efficiency: *Turing machine* signatures, *symbolic constraint* signatures, and *regular expression* signatures. A Turing machine signature can be precise, i.e., no false positives or negatives. However, matching a Turing machine signature may take an unbounded amount of time because of loops and thus is not applicable in all scenarios. Symbolic constraint signatures guarantee that matching will terminate because they have no loops, but must approximate certain constructs in the

```

1 char *url = malloc(4);
2 int c = 0;
3 if(inp[c] != 'g' && inp[c] != 'G')
4     return BENIGN;
5 c++;
6 while(inp[c] == ' ') c++;
7 while(inp[c] != ' '){
8     if(c >= 4) return EXPLOIT;
9     *url = inp[c]; c++; url++;
10 }
11 return BENIGN;

```

Figure 2. The TM signature for our running example

program such as looping and memory aliasing, which may lead to imprecision in the signature. Regular expression signatures are the other extreme point in the design space because matching is efficient but many elementary constructions such as counting must be approximated, and thus the least accurate of the three representations.

Turing machine signatures. A Turing machine (TM) signature is a program T consisting of those instructions which lead to the vulnerability point with the vulnerability condition algorithm inlined. Paths that do not lead to the vulnerability point will return BENIGN, while paths that lead to the vulnerability point and satisfy the vulnerability condition return EXPLOIT.³ TM signatures can be precise, e.g., a trivial TM signature with no error rate is emulating the full program. A TM signature for our running example is given in Figure 2:

Symbolic constraint signatures. A symbolic constraint signature is a set of boolean formulas which approximate a Turing machine signature. Unlike Turing machine signatures which have loops, matching (evaluating) a symbolic constraint signature on an input x will always terminate because there are no loops. Symbolic constraint signatures only approximate constructs such as loops and memory updates statically. As a result, symbolic constraint signatures may not be as precise as the Turing machine signature.

Let $x:y$ represent an inclusive range, e.g., $\text{inp}[1:5]$ means input bytes 1 through 5, inclusive. Then the symbolic constraint signature (after considerable simplification for readability) for our running example is given in Figure 3.

This signature states that the ten-byte input matches the signature if the first input byte is 'G' or 'g', followed by anywhere from 0 to 4 space characters, followed by at least

³A path in a program is a path in the program's control flow graph.

```

(inp[0] = 'g' ∨ inp[0] = 'G') ∧
[(input[1:5] != ' ') ∨
(inp[1] = ' ' ∧ inp[2:6] != ' ') ∨
(inp[1:2] = ' ' ∧ inp[3:7] != ' ') ∨
(inp[1:3] = ' ' ∧ inp[4:8] != ' ') ∨
(inp[1:4] = ' ' ∧ inp[5:9] != ' ')]

```

Figure 3. The symbolic constraint signature for our running example.

5 non-space characters. At least 5 non-space characters are needed in order to overflow the 4-byte allocated `url` buffer. Note this signature is created by unrolling the loops on lines 8-9 and 10-12 of the TM signature. Although in our example we can statically infer how many times to unroll the loop, in general such inferences are not possible and an upper bound to unroll loops must be provided (this is the same approach taken by bounded model checkers [15]).

Regular expression signatures. Regular expressions are the least powerful signature representation of the three, and may have a considerable false positive rate in some circumstances. For example, a well-known limitation is regular expressions cannot count [26], and therefore cannot succinctly express conditions such as checking a message has a proper checksum or even simple inequalities such as $x[i] < x[j]$. However, regular expression signatures are widely used in practice. The regular expression signature we would produce for our running example (using the data-flow techniques described in Section 3.4) is

$[g|G][\]^*\wedge\{5\}$, which matches any input that begins with 'g' or 'G', followed by zero or more spaces, followed by at least 5 or more (represented as $\{5\}$) non-space characters.

Other signature types. One of the main contributions from our construction is any language class may be used to represent a signature. The signature user is free to pick the appropriate representation for their situation. We leave as future work systematic and formal investigation into other signature representations, e.g., context free languages.

2.4 Signature Operations and Efficiency

We summarize upper bounds for various signature operations in Table 1. Due to space constraints, we prove these bounds in the extended version of this paper [8]. The vulnerability language $L_{\mathcal{P},c}$ is recognized by a vulnerability signature representation via the MATCH operation. Matching efficiency is likely a primary concern when picking a signature representation. Turing machine signature

Representation	Creation	Signature Size	Matching	Minimization	Equivalence
Turing machine Sig.	poly(N)	poly(N)	Undecidable	Undecidable	Undecidable
Symbolic Constraint Sig.	poly(N)	poly(N)	poly(S)	exp(S)	exp(S)
Regular Expression Sig.	poly(N) - exp(N)	exp(N)	O(S)	O(S^2)	O(S^2)

Table 1. Summary of approximate bounds for the three vulnerability signature representations we consider for a program of length N and signature size S . $\text{poly}(X)$ denotes a function polynomial in X , and $\text{exp}(X)$ denotes a function exponential in X .

matching is undecidable (since matching can be reduced to the halting problem), and symbolic constraint signatures matching can be done in polynomial time. Regular expression matching can be performed in linear time.

TM signatures are created by encoding and inlining the vulnerability condition, which takes polynomial time. Symbolic constraint signature generation requires first creating a TM signature, then several additional polynomial-time transformation such as unrolling loops a fixed number of times. Regular expression signature creation entails either solving the symbolic constraint signature, which may take exponential time (in fact, is PSPACE-complete [8]), or performing data-flow analysis on the original program, which takes polynomial time. The former accurately represents all solutions to the symbolic constraints, while the latter approximates the original program via data-flow analysis and is less accurate (see Section 3.4).

Signature *merging* is another important operation. In our model, merging signatures A and B is equivalent to performing a single analysis of $L_{ab} = L_a \cup L_b$, that is, the union of the languages for both vulnerabilities. The union operation for TM signatures is done by creating a new condition $c_{ab} = c_a \vee c_b$ that evaluates true if $T(\mathcal{P}, x) \models c_a$ or $T(\mathcal{P}, x) \models c_b$. The union operation for symbolic constraints is the disjunction of the individual constraints, i.e., either constraint system could be satisfied. The union operation for regular expression is the “or” ($\mid$) operator.

2.5 Monomorphic Execution Path (MEP) and Polymorphic Execution Path (PEP) Signature Coverage

We introduce the notion of vulnerability signature coverage in which we create a vulnerability signature with respect to only a subset of program paths an exploit may follow. The ability to consider subset of paths to a vulnerability (as opposed to all program paths an exploit may follow) is important since creating a signature for all program paths that lead to the vulnerability may be too expensive. Our signature creation techniques take an iterative approach in order to be scalable where we successively improve signatures by first considering a small coverage, and then incrementally increasing our coverage to include more program paths to

the vulnerability.

First, consider a single path in the program an input may take that satisfies the vulnerability condition, which we call *Monomorphic Execution Path* (MEP) coverage. Our initial MEP path is usually the path taken by the sample exploit. An MEP covers only those program instructions executed by an exploit on a single path to the vulnerability point, excluding statements with no effect on the computation, e.g., line 6 in the sample exploit is semantically a no-op with respect to the vulnerability. Within an MEP, for each conditional branch encountered, one target is an instruction leading towards the vulnerability point, while the other target is a state BENIGN. An MEP is therefore a straight-line program. At the vulnerability point the vulnerability condition is evaluated, which returns either BENIGN or EXPLOIT. The vulnerability signature consists of all inputs that reach the EXPLOIT state. Note that straight-line programs do not imply that only a single input leads to the vulnerability point: there usually exists many other inputs $x' \neq x$ that both reach the vulnerability point and the vulnerability condition evaluates to EXPLOIT. For example, exploits usually have a payload which executes arbitrary attacker code. A straight line program will return EXPLOIT for exploits with different payloads because the execution of different variants only differ *after* the vulnerability condition has been satisfied.

A *Polymorphic Execution Path* (PEP) coverage includes many different paths (i.e., MEPs) to the vulnerability point. A *complete* PEP coverage includes all paths to the vulnerability point. Therefore, a complete PEP coverage signature accepts all inputs $\in L_{\mathcal{P},c}$, i.e., the signature is complete. More formally, complete coverage is obtained by generating a signature for a *chop* [27, 48] of the program, which includes all instructions that may be executed between a `read` statement where an exploit may be read in and the vulnerability point. A chop has two distinguished nodes: v_{init} and v_{final} . v_{init} corresponds to the input `read` statement (if multiple input `read` statements exist, then v_{init} is an abstract node that is connected to each `read` statement in the control flow graph). v_{final} corresponds to the inlined vulnerability condition branch returning EXPLOIT. We outline our algorithm for computing the chop in Section 3.2.

In our signature-creation algorithm, we initially begin with the MEP path consisting of those instructions executing in the exploit trace T . We then compute a program chop of the vulnerability, where v_{init} is the initial read of the sample exploit, and v_{final} is the vulnerability point. The chop contains all possible execution paths from where an exploit was read (in the trace) to the vulnerability point. We then initially create a signature S for the MEP path given by the execution trace, and then iteratively improve S by considering other paths.

For our running example, the MEP coverage consists of the instructions executed in the trace. The complete PEP coverage consists of lines 1-12, excluding line 6.

3 Automatic Vulnerability Signature Creation

At a high level, our algorithm for computing a vulnerability signature for program $\mathcal{P}$, vulnerability condition c , a sample exploit x , and the corresponding instruction trace T is depicted in Figure 4. In this section we detail how we perform each of the steps:

1. Pre-process the program before any exploit is received by:
 - (a) Disassembling the program $\mathcal{P}$ (Section 3.1).
 - (b) Converting the assembly into an intermediate representation (IR) (Section 3.1).
2. Compute a chop with respect to the trace T . The chop includes all paths to the vulnerability point including that taken by the sample exploit (Section 3.2).
3. Compute the signature:
 - (a) Compute the Turing machine signature (Section 3.3.1). Stop if this is the final representation.
 - (b) Compute the symbolic constraint signature from the TM signature (Section 3.3.2). Stop if this is the final representation.
 - (c) Compute the regular expression signature from the symbolic constraint signature (Section 3.4).

3.1 Disassembling the Binary Program and Converting to the IR

We first disassemble the binary and identify function boundaries. We do not require the symbol table as functions can be identified via their prologue and epilogue. Next, we convert the disassembled instructions into an intermediate representation (IR). The IR disambiguates instructions by making implicit hardware side-effects explicit. Although this step is seemingly straight-forward, it is actually fairly involved. The main complication we address is modern architectures such as x86 implicitly set and test hardware registers, which can affect program execution, i.e., these tests

and sets do not appear explicitly in the assembly. For example, the overflow flag may be automatically set when executing arithmetic operation, then later tested by a conditional jump. Another complication is the same register may be indexed in different modes, e.g., `al` is the lower 8 bits of the `eax` register, so any instruction affecting `al` must simultaneously affect `eax` in the IR.

More concretely, the x86 instruction set contains over 60 instructions that perform via hardware test or set operations on the `EFLAGS` register. Extra IR statements must be added to almost all operations to reflect the updates done in hardware. Worse, which statements to add is specific to the particular mode of the operands. The x86 architecture has 8-bit mode, 16-bit mode, etc., which is set depending upon the format of the instruction operands. For example, `add %ax, %bx` is an addition in 16-bit mode since the registers specified are 16-bits long. Overflow, the carry flag, and other implicit hardware-assisted effects must then be set with respect to 16-bits. A very similar instruction `add %eax, %ebx` is 32-bit mode, and implicit hardware effects must be done with respect to 32-bits.

We perform the remaining steps — program chopping and vulnerability signature creation — on the IR statements.

3.2 Computing the Chop on the IR

We first compute the chop [27, 48] of the vulnerability with respect to the exploit x and trace T (as discussed in Section 2.5). Note our chopping algorithm results in an imprecise chop because we lack pointer analysis. The result of the chop is a smaller program $\mathcal{P}'$ in which every path begins at the `read` statement in the trace and ends at the vulnerability point. We can then select in the signature generation step any set of paths in $\mathcal{P}'$ and compute a signature.

We perform a chop on the program's callgraph. The chop contains all functions that may be executed between reading in the exploit and the vulnerability point. The chop is performed by essentially doing a reachability analysis so that any function in a call sequence that may reach the vulnerability point is included.

A callgraph is a directed graph where each function is a vertex, and edges represent the caller-callee relationship of functions. We perform the following algorithm on the callgraph to create the chop given start IR statement v_{init} , which is the `read` statement for the exploit in the trace, and the vulnerability point v_{final} in the trace T . Let F_{init} and F_{final} be the functions enclosing the v_{init} and v_{final} nodes respectively. Note that there is at least one path from F_{init} to F_{final} : the one that appears in the trace. We then add an extra edge from F_{final} to F_{init} , resulting in a loop in the callgraph. We then calculate the strongly connected component (SCC) containing F_{init} and F_{final} . This SCC is the chop, since it contains all reachable functions from F_{init} to F_{final} .

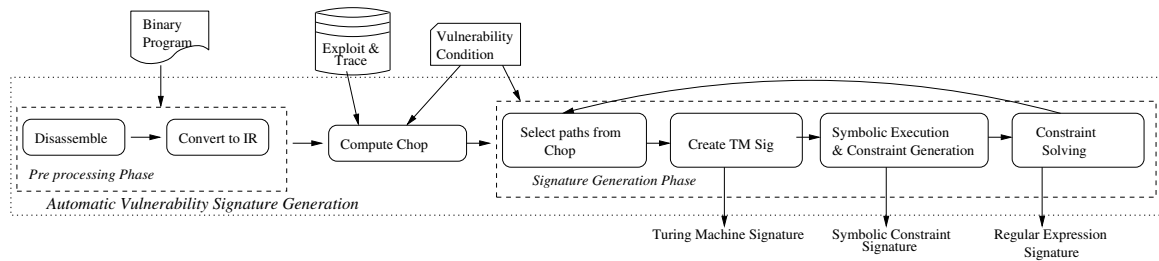


Figure 4. A high level view of the steps to compute a vulnerability signature.

One problem we must deal with at the binary level is the widespread use of indirect jumps, e.g., `jmp %eax` is indirect while `jmp 0x80bebefa` is direct. Note some indirect jumps correspond to source code constructs such as function pointers, while some are compiler-generated as optimizations. The central issue is a target of an indirect jump could potentially be any other instruction. As a result, any control flow graph (including dependency graphs) would have an edge from each indirect jump to all other instructions.

In order to deal with the widespread indirect jumps in binaries without pointer analysis, when creating the callgraph we make the target of each indirect jump a special node IJMP. Our algorithm for computing the chop then will essentially ignore indirect jumps until a chop is computed. After computing the chop, we constrain each indirect jump so that the target is within the chop. One limitation of this approach is that technically we could be incorrectly excluding a function that only appears as the target of an indirect jump. The indirect jump problem may or may not disappear once we have implemented function pointer analysis. It remains unclear (and a point of future work) how precisely such analysis will be able to pin down the targets of indirect jumps.

3.3 Computing the Signature

We compute the signature with respect to the chop. We compute a PEP signature by iteratively considering single MEP paths (except in our data-flow analysis optimization). Our iterative method works because we can pick any path or set of paths within the chop, perform our analysis, and output the corresponding vulnerability signature. The complete PEP coverage signature (Section 2.5) is then the analysis of all paths in the chop. We begin by describing how we compute the complete PEP TM signature, which in turn becomes input to symbolic constraint and regular expression signature generation.

```

1 char *url = malloc(4);
2 int c = 0;
3 if(inp[c] != 'g' && inp[c] != 'G')
4     return BENIGN
5 c++;
6 if(inp[c] != ' ') return BENIGN;
7 c++;
8 if(inp[c] == ' ') return BENIGN;
9 *url = inp[c]; c++; url++;
10 if(inp[c] == ' ') return BENIGN;
11 *url = inp[c]; c++; url++;
12 if(inp[c] == ' ') return BENIGN;
13 *url = inp[c]; c++; url++;
14 if(inp[c] == ' ') return BENIGN;
15 return EXPLOIT;

```

Figure 5. The MEP TM signature for our running example.

3.3.1 Turing Machine Signature Generation

MEP Turing machine signature generation. Our initial MEP Turing machine signature is created with respect to the path followed in the instruction trace. Therefore, the initial signature will match the sample exploit, and certain exploit variants such as changing the exploit payload. We create the initial MEP TM signature by reading in the instruction trace and including the corresponding IR statements.

Sequential instructions in the trace correspond to sequential statements in the MEP Turing machine signature. Conditional branch statements have exactly two targets in the IR, which during signature creation are either BENIGN or EXPLOIT. Any branch that does not lead towards the vulnerability point returns BENIGN. EXPLOIT is only returned if the vulnerability point is reached and the vulnerability condition is satisfied.

We encode the vulnerability condition as a function. At the vulnerability point we insert a jump to this function, which does a final check to see if the program is in the vulnerable state, and returns EXPLOIT if satisfied else BE-

NIGN. Figure 5 shows the MEP vulnerability signature we would return for our running example with the vulnerability check inlined.

PEP Turing machine signature generation. A PEP Turing machine signature is created similar to an MEP Turing machine signature. The PEP signature first computes the chop, and then computes which jump targets cannot lead to the vulnerability point via standard graph reachability analysis. Paths that terminate or cannot lead to the vulnerability point return BENIGN. We also again insert a call to the vulnerability condition function at the vulnerability point, which returns either BENIGN or EXPLOIT. Figure 2 shows the complete PEP vulnerability signature with the vulnerability condition inlined.

3.3.2 Symbolic Constraint Signature Generation

A symbolic constraint signature is a set of constraints an exploit of the vulnerability must satisfy. We use the TM signature as the input to symbolic constraint signature generation, and at a high level generate constraints that represent meeting the correct conditionals in the TM to reach the vulnerability point and satisfy the vulnerability condition. The symbolic constraint signature is an approximation of the TM signature because we may have to statically estimate the effects of loops and memory updates as constraints on the input. The symbolic constraint system is built up by symbolically evaluating the TM signature program on symbolic inputs instead of actual inputs (values).

More formally, we build up the constraints based upon symbolically executing paths in the TM. Each function in the TM signature is represented by a control flow graph (CFG), which is a direct graph $(V, E, v_{entry}, v_{exit})$ where each IR instruction is a node in V , each transfer of control between instructions is an edge in E , and v_{entry}, v_{exit} are distinguished entry and exit nodes. Conditionals in the control flow graph become constraints to take the appropriate branch to reach the vulnerability point and satisfy the vulnerability condition.

Single Static Assignment (SSA) form. We must convert the IR into a single static assignment (SSA) [41] form prior to symbolic constraints (this step can be performed during the pre-processing phase). Normally, memory locations and registers are destructively updated many times in the lifespan of a program, e.g., $x = x + 1$ destructively updates the x on the right-hand side (RHS) when assigning to x on the left-hand side (LHS). However, symbolic execution requires each variable be treated as a single logical entity that is assigned to only once. SSA form is a semantically equivalent form of the program which satisfies this criteria.

The SSA form of sequential statements is just a unique renaming of each LHS. For example, $x = x + 1$ becomes $x_2 = x_1 + 1$. For control statements, SSA introduces a special assignment called ϕ -functions which merges several possible definitions of a variable into one. For example, the if-then-else statement

```
if (x < 2) z = 10; else z = 20;
```

becomes

```
if (x0 < 2) z1 = 10 else z2 = 20;
z3 =  $\phi(z_1, z_2)$ ;
```

where z_3 is assigned z_1 on the true branch and z_2 on the false branch.

MEP symbolic execution. We perform MEP symbolic execution by evaluating the MEP TM signature. Recall that the MEP TM signature is a straight-line program. Then there is a single path $\pi = v_{entry}, v_1, \dots, v_{exit}$ that goes through the vulnerability point and the vulnerability condition. All other paths will end up returning BENIGN and need not be considered. The result of symbolic execution on π is a set of constraints on input variables that when met results in an execution from v_{entry} , through the vulnerability point and the inlined vulnerability condition to v_{exit} .

We begin by creating symbolic input variables $i_0, \dots, i_n$ where n is the length of the symbolic input to consider, e.g., n is initially the length of the sample exploit x . Each statement is then executed on these inputs, resulting in a symbolic formula at each step. There are three fundamental operation types to evaluate symbolically: memory updates, arithmetic operations, and branch predicate evaluation. Symbolic execution of arithmetic operations is simply a substitution procedure. For example, $x = a + i_0; y = x * z$ becomes $y = (a + i_0) * z$.

A memory store operation is an assignment of a value to a symbolic memory location (stack and heap assignments are handled in a uniform fashion). We adopt a model similar to UCLID [9] for handling memory updates. The initial state of M is given by m_0 . Reads and writes are modeled as λ expressions, where a write to memory location A with value D yields a new M' :

$$M' = \lambda addr. ITE(addr = A, D, M[addr])$$

The result of a write is an if-then-else (ITE) λ expression. A subsequent read behaves as follows: the address to read is applied as the argument to the write λ expression. If the supplied address matches A , then D is returned, else we recurse to the next memory address given by $M[addr]$.

Without loss of generality, we assume each branch predicate (such as j_e (jump if equal) or j_z (jump if zero)) $v_i \in \pi$ evaluates to true in order to create the desired total path π . A branch predicate forms an arithmetic constraint (with some

expressions perhaps involving memory reads and writes) relating the symbolic execution to some constant, e.g., `jz y` where $y = (a + i_0) * z$ as before and `jz` is “jump if zero” results in the constraint $(a + i_0) * z = 0$. Constraints evaluate to a constant because machine instructions only allow comparison of an expression to a constant. The total symbolic formula is then just the conjunction of each branch predicate.

The constraint system consisting of the conditions on each branch predicate in π is returned as the desired signature. Optionally, constraint systems can be simplified, which consists of deducing how multiple constraints can be collapsed into a single constraint.

PEP symbolic execution. PEP symbolic execution is similar to the MEP case, except we must deal with loops. Loops are handled by computing fixed points. However, in data-flow analysis a widening operator is used to guarantee that the iteration to compute the fixed-point terminates [6, 18]. Currently, we use the following algorithm to handle loops:

- First, we identify induction variables [41, Chapter 14] in each loop. For example, the induction variable for the first `while` loop in Figure 1 is `c`. We also compute the bounds on the induction variable, e.g., the bound on the induction variable `c` is $c \geq 1$.
- Assume that an induction variable is used to index the input array in the condition used in the `while` loop. The condition used in the `while` loop along with the bounds on the induction variable gives us the desired result. For the first `while` loop in Figure 1 the condition that is generated is

`(input[c] = ' ') where (c >= 1)`

3.4 Regular Expression Signature Generation

3.4.1 Computing MEP Regular Expression Signatures

One method for generating a regular expression is to solve the constraint system S to a set $x : x \in S$ and or-ing ($\mid$) together all members, e.g., if $S = \{00, 01, 21\}$, then the regular expression is `00|01|21`⁴. This method is explored heavily in test-case generation literature [10, 22, 23, 24, 25]. We adopt this approach to our problem setting.

Divide-and-conquer. The number of variables to consider within a single path may be very large, e.g., millions of variables at the assembly level. We address this

⁴A reader may notice this expression is precise, and wonder when the solution will not be precise. The answer is as precise as the symbolic representation, e.g., if the symbolic representation only unrolls a loop once, then the regular expression signature will not reflect inputs that may cause the loop to be executed more than one time.

problem by decomposing an MEP single-path solution into smaller sub-paths we can consider independently. Let $\pi = v_{entry}v_1 \dots v_{exit}$ be an MEP path. A sub-path is a sequence of instructions $\pi_i = v_i v_{i+1} \dots v_\ell \in \pi$ in the CFG. A sub-path π_i can be independently evaluated with respect to another sub-path π_j if no computation in π_i could ever affect a computation in π_j , and vice-versa. Formally, we partition $\pi = \pi_1 \pi_2 \pi_3 \dots \pi_j$, where each π_i is a partition with no data dependencies with another sub-path π_j . A data dependency exists between π_i and π_j if π_i computes a value that π_j uses.

Since no computation in one sub-path π_i could affect a computation in another sub-path π_j , each sub-path can be independently solved, then the final solution can be combined. The solution to each sub-path π_i is computed as above by solving the corresponding constraint system for the sub-path. The full path π is then the conjunction ($\wedge$) of all sub-paths.

MEP solution. Our approach allows us to divide a single MEP into possibly several smaller sub-problems. Let the MEP path $\pi = \pi_1 \pi_2 \dots \pi_n$ correspond to evaluating the symbolic input in order $i_1, \dots, i_n$. Since sub-paths are independent, we can always reorder the sub-paths so this is the case. Then the signature for an MEP is the concatenation of the solution for each sub-path. If π_i has solution S_i , then the resulting signature is $S = S_1 S_2 \dots S_n$.

3.4.2 Computing PEP Regular Expression Signatures

We consider two approaches for computing a PEP solution. The first method considers each MEP path within a PEP independently, and solves the symbolic constraints exactly. The second method is an optimization based on data-flow analysis which can be applied to portions of the PEP control-flow graph when certain conditions stated below are met. The data-flow analysis optimization works on basic blocks instead of paths and does not require access to a constraint solver.

Exact PEP solution. The PEP solution iteratively explores paths, and then solves them as an MEP solution. We note that in practice one would likely create an initial MEP signature for the sample exploit, then process other paths in the background. This approach generates an initial narrow signature quickly, then continues to refine it as we perform more analysis.

PEP data-flow optimization. In many cases we may be able to determine: (a) the data dependencies partition a vulnerability into two or more components (w.r.t. the CFG), and (b) some of these components do direct comparisons

with input values. For example, many protocols have keywords or have constant values for specific fields which the input is simply compared against.

We use data-flow analysis to efficiently compute the language accepted by such components. Since each component has no data dependencies with other components, the solution to each component can be inlined into the complete PEP or MEP solution. At a high level, data-flow analysis iteratively processes a CFG until a fixed point of data-flow facts is reached. Data-flow analysis is widely used in compilers and is highly efficient.

The data-flow analysis combines regular expressions accepted for each basic block (i.e., a block of contiguous instructions with a single entry and exit point) into a regular expression accepted by the entire component. Due to space constraints, we give here a very rough overview of data-flow analysis and leave further discussion to the extended version of this paper [8]. At a high level, each CFG edge is labeled with a set of data-flow facts, which in our case is the regular expression accepted by the basic block for true edges, and the negated regular expression for false edges. One key component for a data-flow analysis is specifying a $\sqcap$ (meet) operator, which summarizes how multiple incoming edges to a node are combined, i.e., combining the regular expression for a point of confluence of incoming edges. Our $\sqcap$ operator states how to combine regular expressions α and β at a confluence point, e.g., if $\beta = !\alpha$, then the confluence point corresponds to the regular expression Σ . In our running example, the instructions on line 4 can be analyzed independently using data-flow analysis, resulting in the regular expression $g|G$ for the first byte of the input. Note realistic programs usually have much larger components than in our example which are amenable to data-flow analysis.

4 Evaluation and Implementation

We have implemented a prototype system to evaluate our techniques for automatically generating signatures. In this section we briefly discuss implementation details of our prototype, and then present our evaluation results. Our evaluation results show that even an MEP vulnerability signature is of far higher quality than signatures generated with previous approaches. We focus on creating regular expression signatures since they require generation of the Turing machine and symbolic signature.

4.1 Implementation

Our total prototype for implementing our techniques is about 9000 lines of C++ code. We currently use CBMC [16], a bounded model checker, to help build and

solve symbolic constraints to produce regular expression signatures.⁵

Disassembling the program, converting to IR, and obtaining instruction traces. Our binary program disassembler is based upon Kruegel et. al. [34]. We then translate each instruction into the appropriate IR statement via our own translation language.

Instruction traces can be efficiently generated for most modern architectures including x86 via hardware [5, 49] or via software [1, 37, 42]. An instruction trace contains the instruction address and optionally the value of the operands for each instruction executed. Although the number of instructions executed may be large, the corresponding trace can be efficiently represented [38, 53]. We currently use Pin [37] to create our traces.

Solving the constraint system. We use model checking to solve the system of constraints. We translate the constraints into constraints on C variables and use CBMC [16]). We then assert to the model checker that the vulnerability condition is unsatisfiable. The model checker will either verify the vulnerability condition is unsatisfiable, or solve the constraint system and present a counter-example which, by construction, is a satisfying input. This process can be iterated to exhaustively enumerate all possible satisfying inputs (i.e., exploits). The regular expression signature is the “or” of all satisfying inputs. However, this process may be slow when an input byte may be any of the 2^{256} values. Therefore, we currently apply a widening operator such that any byte that appears to be unconstrained after 3 iterations becomes a wild-card byte. The widening step may introduce false positives, and can be eliminated when desired.

We show that precise regular expression signature generation can be reduced to the model checking problem in the extended version of this paper [8]. Exploring less precise generation techniques, as well as techniques that work on practical examples but may be theoretically limited, is an area of future work.

Implementation limitations. Our current implementation is a prototype used for researching automatic signature generation. Although our prototype works in our research setting, there are a number of limitations. As mentioned previously, alias analysis is currently not supported. Specifically, we assume that no two memory locations are aliases. In addition to the possible imprecision this may introduce during symbolic execution, this limitation prevents us from computing a true chop [27, 48]. Our current callgraph-based chopping algorithm is less precise than a true chop,

⁵We do not use source code despite the fact this is primarily a C model checker.

which primarily results in larger MEP and PEP coverages than necessary. Second, we currently create sub-paths based upon control-flow based analysis, which may not accurately identify when two sub-paths are independent (Section 3.4). Finally, our IR transformations do not handle floating point operations, and we currently do not support the entire x86 instruction set (we add operations as needed for our experiments). All these limitations are orthogonal to our problem and can be resolved by implementing known techniques. We currently manually verify none of these problems introduce errors into our results.

4.2 MEP Evaluation

4.2.1 ATPhtp

ATPhtp is a webserver written in C [47]. ATPhtp version 0.4b is vulnerable to a common printf-style buffer-overflow when an HTTP request is too long. Specifically, an exploit of the ATPhtp vulnerability must meet the following conditions: (a) the HTTP request method is case-insensitive, and must be either “get” or “head”; (b) the first byte of the requested file name must be ‘/’, and cannot be followed by ‘/’; (c) the requested filename cannot contain the substring “/..” or end with “/..”; and (d) the requested filename must be over 677 characters long.

We use the exploit sample from [46], which consists of the request `GET /`, followed by the shell code, followed by the HTTP protocol string `HTTP/1.1`. In this experiment, the vulnerability condition given for ATPhtp is that no pointer should be able to write to a return address.

Signature result and quality. We generated the symbolic constraints, which were partitioned into 10 distinct sub-paths that were analyzed independently. We solved the constraints and create a regular expression in a little over a second, with the average time per partition taking 0.1216s.

We generated the regular expression signature `[g|G][e|E][t|T][ ]/.{423}///.{3}/.{386}`. This regular expression is almost perfect w.r.t. the necessary conditions to reach the vulnerability as stated previously. In particular, it recognizes that the `get` keyword is case insensitive, and that most bytes can be anything. The bytes that are constraints (“/” and “//” in the signature) are both contained in the exploit and explicitly tested along the MEP vulnerability path that the exploit took. We contrast our signature with previous exploit-specific signature generation approaches [44, 43, 36], which at best only identify small parts of our signature and do not match different exploit variants such as those that crashes the server instead of injecting code. Our signatures will catch all exploit variants given only a single exploit sample.

4.2.2 BIND

BIND is one of the most popular DNS servers. BIND supports a secret key transaction authentication mechanism where messages are signed with a transaction signature (TSIG) [55]. BIND 8.2.x is susceptible to a stack overflow vulnerability in the TSIG processing code.

The attacker must send a valid DNS transaction signature request in order to exploit this vulnerability [12]. DNS is a binary-based protocol in which all messages are struct-like. DNS (and the exploit) can be TCP or UDP-based, though here we only consider the UDP protocol messages. DNS messages begin with a header, followed by a number of resource records (RR). An exploit of this vulnerability must satisfy the following conditions: (a) the request must be a query, which is represented by byte 2 of the message being 0; (b) there must be questions present, meaning that the field specifying the number of questions (byte offsets 4 and 5) must be greater than zero, and that there must be properly encoded questions starting at offset 12; (c) the field specifying the number of additional resource records (byte offsets 10 and 11) must be greater than zero; (d) The DNS must contain a resource record with the type field set to TSIG, which is 0x00af. Since DNS may have many different resource records in a single request, the specific byte offset for this field is a function of several other fields in the request. We use the TSIG vulnerability exploit from the LION worm [54] as our sample exploit.

Signature result and quality. We generated the symbolic constraints, which again could be partitioned into 10 distinct graphs, which we independently analyzed. The generated regular-expression signature specified that bytes 6-10 must be zero, that bytes 268 and 500, which indicate the end of each query in the exploit, must be 0, that byte 12 must not be 0, which is the first byte of the first query, and finally, that bytes 505 through 507 must be 0x0000fa, which is the 0 byte at the beginning of the additional resource records section, followed by the field type TSIG. We verify that the constructed signature identified all constraints that must be met to exploit the vulnerability. We also verified the false-positive rate of our signature by matching it again 1,000,000 DNS requests (trace taken from a high-traffic DNS server that serves several top level domains). There were no false positives.

4.3 PEP Evaluation

The chop of ATPhtp took 30 μ s and found 88% of all functions were reachable between accepting a connection and the vulnerability point (including all libraries). As mentioned previously, one technique for generating a PEP signature is to consider each MEP path independently. Another technique is to estimate the effects of multiple paths

simultaneously. Our current prototype implementation for the latter technique is limited to moderate-sized functions. Unfortunately, the ATPhTtpd and BIND vulnerabilities use extremely large library function which consists of several thousand basic blocks. Addressing scalability issues is an important part of our future work. We expect existing state reduction techniques from model checking will help solve this problem.

Here, we evaluate our PEP techniques on synthetic examples. We compile down our running example to a binary, and then calculate the full PEP solution. The regular expression generated is $[g|G][]^*[\wedge_] \{5, \}$. The total time to compute the answer is about 1.5 seconds. Alternatively, our tool can also produce the regular expression for each independent component of the PEP, and then use data-flow facts to produce the final signature. In this setting, our tool runs slightly faster as it does not have to perform symbolic evaluation along all possible paths.

5 Discussion

We provide more extensive analysis, including proofs of the hardness of signature creation and of our data-flow framework in the extended version of this paper [8].

Other application scenarios. At a high level, our techniques generate an input string that reaches a given instruction in the binary. Several other applications of our techniques that we plan on investigating include:

- Improve existing pattern-extraction signature generation algorithms. The quality of a signature generated by pattern-extraction techniques generally improves as the number of exploit samples increase. Our techniques can be used to iteratively generate a new exploit sample x' that is different than the sample exploit x . In this scenario, we can give x' to the pattern-extractor as a *labeled* exploit, which it then uses to improve an existing signature. Note that in previous scenarios pattern extraction would be limited to only x . In addition, we may be able to label tokens within x which may further help the analysis. Finally, we note that our analysis could also be used to help defend against “red-herring” and “coincidental token” attacks.
- Perform robust vulnerability identification. Often it is not known whether a *known* bug is exploitable. Here, the developer would set v_{init} to the appropriate `read` statement and v_{final} to the line for the bug. Our techniques will generate a sample exploit when possible, confirming whether a bug is exploitable or not.
- Vendor patches often miss all possible paths to a vulnerability. Missing alternate paths is not only a security problem, but can also be an embarrassment to the vendor because even “patched” systems may still be

compromised [11]. Our techniques can be adapted to see if a given patch covers all possible ways a vulnerability may be exploited.

Complete but unsound signatures. Every satisfying solution to the generated symbolic equations is an exploit string, thus the signature is sound but not complete. A complete but potentially unsound signature, i.e., no false negatives but false positives, can be created by setting the initial signature to Σ^* and removing any input that leads to BE-NIGN state.

Identifying sources of signature imprecision. Our construction allows a signature creator to tune accuracy and generation time in several ways. First, the creator has a choice of signature representations. Second, the creator can choose how much information to retain for less expressive representations. For example, when creating a symbolic representation the creator may choose how many times loops are unrolled. Third, the creator can choose how much analysis to perform. For example, when creating a regular expression signature theorem proving can be employed to enumerate every input string that may exploit the program, or faster but less accurate data-flow analysis. We believe these choices allow a creator to gain a fundamental understanding of the overall accuracy of the final generated signature by comparing their generated signature to the perfect TM signature.

6 Related Work

Signature creation. In Section 1 we detailed most previous work in this area. Here we mention that Vigilante has independently proposed signatures which are essentially straight-line programs, not regular expressions [17], much like our MEP symbolic constraint signatures. However, Vigilante only creates a signature for the execution path taken by the sample exploit, and does not explore more extensive coverages or other vulnerability signature representations.

Estimating language classes. A significant part of creating a vulnerability signature boils down to conservatively estimating the higher-powered language such as a Turing machine with a lower-power language such as a regular expression. Our techniques provide one way of accomplishing this. For example, Mohri and Nederhof present an algorithm for converting certain context-free languages into regular expressions [39]. We are unaware of other significant work in this area.

Program analysis. We use many static analysis techniques such as symbolic execution [31], abstract interpretation [18], model checking [15], theorem proving [20], data-flow analysis [29], and program slicing [57]. Each of these

areas is an active research area in which we can benefit from new or more advanced techniques. It would be impossible to note all related work in static analysis; the reader is referred to [4, 41] for an overview of the subject.

Automatic test case generation research explores the problem of automatically creating an input that reaches a particular point in the program [10, 22, 23, 24, 25]. We are interested in a very similar problem where we want to approximate *all* inputs that reach a certain location. Also, our problem setting is relaxed since we may tolerate signatures with false positives and/or negatives.

Another closely related area is static analysis of program generated string expressions. This line of work aims at discovering possible strings *generated*, as opposed to accepted by a program. Christensen *et al.* performed string analysis on Java programs where type information is available [13]. Christodorescu *et al.* extended Christensen's work to x86 binaries [14]. These techniques are exciting, though more research is needed to apply their techniques to our problem setting. In particular, this approach only handles strings and not other types such as integers.

7 Conclusion

We presented a general framework for obtaining a new type of signature called vulnerability signatures. Given a single sample exploit, we presented techniques for automatically generating a signature of higher quality than previous approaches. In addition, our formulation opens up a wide variety of signature representations. In particular, we discuss three distinct types of vulnerability signature representations: Turing machine, symbolic constraints, and regular expressions. We provide theoretical and practical insights into these three signature representations. We conclude that our approach is promising alternative to exploit-centric techniques.

References

- [1] Dynamorio. <http://www.cag.lcs.mit.edu/dynamorio/>.
- [2] K2, admmtute. <http://www.ktwo.ca/c/ADMmtute-0.8.4.tar.gz>.
- [3] Metasploit. <http://metasploit.org>.
- [4] A. Aho, R. Sethi, and J. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley Publishing Company, 1986.
- [5] P. Bosch, A. Carloganu, and D. Etiemble. Complete x86 instruction trace generation from hardware bus collect. In *23rd IEEE EUROMICRO Conference*, 1997.
- [6] F. Bourdoncle. Efficient chaotic iteration strategies with widenings. In *Proc. of the International Conference on Formal methods in Programming and their applications (LNCS 735)*, 1993.
- [7] D. Brumley, L.-H. Liu, P. Poosank, and D. Song. Design space and analysis of worm defense systems. In *Proc of the 2006 ACM Symposium on Information, Computer, and Communication Security (ASIACCS)*, 2006. CMU TR CMU-CS-05-156.
- [8] D. Brumley, J. Newsome, D. Song, H. Wang, and S. Jha. Towards automatic generation of vulnerability-based signatures. Technical Report CMU-CS-06-108, Carnegie Mellon University, 2006.
- [9] R. E. Bryant, S. K. Lahiri, and S. A. Seshia. Modeling and verifying systems using a logic of counter arithmetic with lambda expressions and uninterpreted functions. In *Proc. Computer-Aided Verification (CAV)*, 2002.
- [10] C. Cadar and D. Engler. Execution generated test cases: How to make system code crash itself. Technical Report CSTR-2005-04, Stanford, 2005.
- [11] C. Cerrudo. Story of a dumb patch. <http://argeniss.com/research/MSBugPaper.pdf>, 2005.
- [12] CERT/CC. ISC BIND 8 contains buffer overflow in transaction signature TSIG handling code. <http://www.kb.cert.org/vuls/id/196945>.
- [13] A. S. Christensen, A. Möller, and M. I. Schwartzbach. Precise analysis of string expressions. In *Proc. 10th International Static Analysis Symposium, SAS '03*, volume 2694 of *LNCS*, pages 1–18. Springer-Verlag, June 2003. Available from <http://www.brics.dk/JSA/>.
- [14] M. Christodorescu, N. Kidd, and W.-H. Goh. String analysis in x86 binaries. In *Proc. of the 6th ACM Workshop on Program Analysis for Software Tools and Engineering PASTE*, 2005.
- [15] E. Clarke, O. Grumberg, and D. Peled. *Model Checking*. The MIT Press, 1999.
- [16] E. Clarke, D. Kroening, and F. Lerda. A tool for checking ANSI-C programs. In K. Jensen and A. Podelski, editors, *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2004)*, volume 2988 of *Lecture Notes in Computer Science*, pages 168–176. Springer, 2004.
- [17] M. Cost, J. Crowcroft, M. Castro, A. Rowstron, L. Zhou, L. Zhang, and P. Barham. Vigilante: End-to-end containment of internet worms. In *20th ACM Symposium on Operating System Principles (SOSP 2005)*, 2005.
- [18] P. Cousot and R. Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proc. 4th ACM Symposium on Principles of Programming Languages (POPL)*, Jan 1977.
- [19] J. Crandall, Z. Su, S. F. Wu, and F. Chong. On deriving unknown vulnerabilities from zero-day polymorphic and metamorphic worm exploits. In *Proc. 12th ACM Conference on Computer and Communications Security (CCS)*, 2005.
- [20] D. Detlefs, G. Nelson, and J. Saxe. Simplify: A theorem prover for program checking. Technical Report HPL-2003-148, HP Labs, 2003.
- [21] T. Detristan, T. Ulenspiegel, Y. Malcom, and M. V. Underduk. Polymorphic shellcode engine using spectrum analysis. <http://www.phrack.org/show.php?p=61&a=9>.
- [22] P. Godefroid, N. Klarlund, and K. Sen. DART: Directed automated random testing. In *Proc. of the 2005 International Conference on Programming Language Design and Implementation (PLDI)*, 2005.

- [23] A. Gotlieb, B. Botella, and M. Rueher. Automatic test data generation using constraint solving techniques. In *ACM Symposium on Software Testing and Analysis*, 1998.
- [24] A. Gotlieb, B. Botella, and M. Rueher. A clp framework for computing structural test data. In *First International Conference on Computational Logic*, 2000.
- [25] N. Gupta, A. Mathur, and M. L. Soffa. Automated test data generation using an iterative relaxation method. In *ACM SIGSOFT Symposium on Foundations of Software Engineering*, 1998.
- [26] J. Hopcroft, R. Motwani, and J. Ullman. *Introduction to automata theory, languages, and computation*. Addison-Wesley, 2001.
- [27] D. Jackson and E. Rollins. Chopping: A generalization of slicing. In *Proc. of the Second ACM SIGSOFT Symposium on the Foundations of Software Engineering*, 1994.
- [28] M. Jordan. Dealing with metamorphism. In *Virus Bulletin Magazine*, 2002.
- [29] G. Kildall. A unified approach to global program optimization. In *1st ACM Symposium on Principles of Programming Languages (POPL)*, 1973.
- [30] H.-A. Kim and B. Karp. Autograph: toward automated, distributed worm signature detection. In *Proc. of the 13th USENIX Security Symposium*, August 2004.
- [31] J. King. Symbolic execution and program testing. *Communications of the ACM*, 19:386–394, 1976.
- [32] C. Kreibich and J. Crowcroft. Honeycomb - creating intrusion detection signatures using honeypots. In *Proc. of the Second Workshop on Hot Topics in Networks (HotNets-II)*, November 2003.
- [33] C. Kruegel, E. Kirda, D. Mutz, W. Robertson, and G. Vigna. Polymorphic worm detection using structural information of executables. In *Rapid Advances in Intrusion Detection (RAID)*, 2005.
- [34] C. Kruegel, W. Robertson, F. Valeur, and G. Vigna. Static disassembly of obfuscated binaries. In *Proc. of the 13th USENIX Security Symposium*, 2004.
- [35] Z. Liang and R. Sekar. Fast and automated generation of attack signatures: A basis for building self-protecting servers. In *Proc. of the 12th ACM Conference on Computer and Communications Security (CCS)*, 2005.
- [36] M. E. Locasto, K. Wang, A. D. Keromytis, and S. J. Stolfo. FLIPS: Hybrid adaptive intrusion prevention. In *Proc. of the 8th International Symposium on Recent Advances in Intrusion Detection (RAID 2005)*, 2005.
- [37] C.-K. Luk, R. Cohn, R. Muth, H. Patil, A. Klauser, G. Lowney, S. Wallace, V. J. Reddi, and K. Hazelwood. Pin: Building customized program analysis tools with dynamic instrumentation. In *Proc. of 2005 Programming Language Design and Implementation (PLDI) conference*, June 2005.
- [38] A. Milenkovic, M. Milenkovic, and J. Kulick. N-tuple compression: A novel method for compression of branch instruction traces. In *Proc. of the 16th international conference on parallel and distributed computing*, 2003.
- [39] M. Mohri and M.-J. Nederhof. *Robustness in Language and Speech Technology*. Kluwer Academic Publishers, 2001.
- [40] D. Moore, C. Shannon, G. Voelker, and S. Savage. Internet quarantine: Requirements for containing self-propagating code. In *2003 IEEE Infocom Conference*, 2003.
- [41] S. Muchnick. *Advanced Compiler Design and Implementation*. Academic Press, 1997.
- [42] N. Nethercote and J. Seward. Valgrind: A program supervision framework. In *Proc. of the Third Workshop on Runtime Verification (RV'03)*, Boulder, Colorado, USA, July 2003.
- [43] J. Newsome, B. Karp, and D. Song. Polygraph: Automatically generating signatures for polymorphic worms. In *Proc. of the IEEE Symposium on Security and Privacy*, May 2005.
- [44] J. Newsome and D. Song. Dynamic taint analysis for automatic detection, analysis, and signature generation of exploits on commodity software. In *Proc. of the 12th Annual Network and Distributed System Security Symposium (NDSS)*, February 2005.
- [45] B. C. Pierce. *Types and Programming Languages*. The MIT Press, 2002.
- [46] r code. ATPhttpd exploit. <http://www.cotse.com/mailling-lists/todays/att-0003/01-atphttp0x06.c>.
- [47] Y. Ramin. ATPhttpd. <http://www.redshift.com/~yramin/atp/atphttpd/>.
- [48] T. Reps and G. Rosay. Precise interprocedural chopping. In *Proc. of the Third ACM SIGSOFT Symposium on the Foundations of Software Engineering*, 1995.
- [49] P. A. Sandon, Y. Liao, T. Cook, D. Schultz, and P. M. de Nicolas. Nstrace: A bus-driven instruction trace tool for powerpc microprocessors. *IBM Journal of Research and Development*, 41(3), 1997.
- [50] S. Singh, C. Estan, G. Varghese, and S. Savage. Automated worm fingerprinting. In *Proc. of the 6th ACM/USENIX Symposium on Operating System Design and Implementation (OSDI)*, Dec. 2004.
- [51] S. Staniford, V. Paxson, and N. Weaver. How to Own the Internet in your spare time. In *11th USENIX Security Symposium*, 2002.
- [52] P. Szor. Hunting for metamorphic. In *Virus Bulletin Conference*, 2001.
- [53] R. A. Uhlig and T. Mudge. Trace-driven memory simulation: a survey. *ACM Computing Surveys*, 29, 1997.
- [54] US-CERT. Vulnerability note vu#196945 - isc bind 8 contains buffer overflow in transaction signature (tsig) handling code. <http://www.kb.cert.org/vuls/id/196945>.
- [55] P. Vixie, O. Gudmundsson, D. Eastlake, and B. Wellington. RFC 2845: Secret key transaction authentication for dns (TSIG). <http://www.ietf.org/rfc/rfc2845.txt>.
- [56] H. J. Wang, C. Guo, D. Simon, and A. Zugenmaier. Shield: Vulnerability-driven network filters for preventing known vulnerability exploits. In *Proc. of the 2004 ACM SIGCOMM Conference*, August 2004.
- [57] M. Weiser. Programmers use slices when debugging. *Communications of the ACM*, 25:446–452, 1982.
- [58] J. Xu, P. Ning, C. Kil, Y. Zhai, and C. Bookholt. Automatic diagnosis and response to memory corruption vulnerabilities. In *Proc. of the 12th ACM Conference on Computer and Communication Security (CCS)*, 2005.